

CSCI-4208:

Developing Advanced Web Applications

Week 11: Lecture 20

Authentication
Sessions & Tokens

Authentication Overview

1. Introduction
2. Sessions
3. Tokens (JWT)
4. Sessions vs Tokens
5. Options for Authorization in SPAs / APIs
6. Node modules & Sessions
7. Node modules & JWT
8. Node Modules & Authentication

1. Introduction

Why do we need session or token for authentication?

HTTP is stateless. All the client requests are stateless. However, there are situations where we would like our states to be remembered. For example, in an online shop, after we put bananas in a shopping cart, we don't want our bananas to disappear when we go to another page to buy apples. ie. we want our purchase state to be remembered while we navigate through the online shop!

To overcome the stateless nature of HTTP requests, we could use either a session or a token.

1. Introduction

- **authentication**: verifying identity (401 Unauthorized)
- **authorization**: verifying permissions (403 Forbidden)

Username/password scheme

- **stateful** (i.e. session using a cookie)
- **stateless** (i.e. token using JWT / OAuth / other)

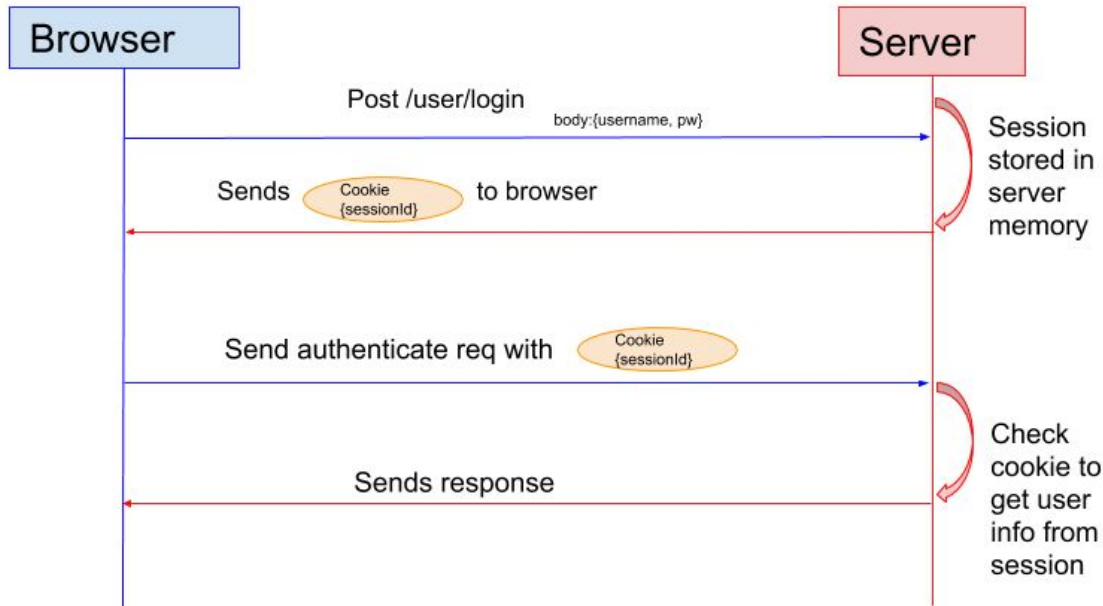
2. Sessions - Flow

Flow

- user submits login *credentials*, e.g. email & password
- server verifies the credentials against the DB
- server creates a temporary user **session**
- server issues a cookie with a **session ID**
- user sends the cookie with each request
- server validates it against the session store & grants access
- when user logs out, server destroys the session & clears the cookie

2. Sessions - Flow

Server creates a session for the user after the user logs in. The session id is stored on a cookie on the user's browser. While the user is logged in, the cookie is sent with every request. The server compares the session id from the cookie against the session information stored in memory to verify user's identity.



2. Sessions - Features

- Every user session is stored server-side (**stateful**)
 - memory (e.g. file system)
 - cache (e.g. Redis or Memcached)
 - DB (e.g. Postgres, MongoDB)
- Each user is identified by a session ID
 - **opaque** ref.
 - no 3rd party can extract data out
 - only issuer (server) can map back to data
 - stored in a cookie
 - signed with a secret
 - protected with flags
- Server-Side Rendering web apps, frameworks (Spring, Rails, Express), scripting langs (Node)

2. Sessions - Cookies

- `Cookie` header, just like `Authorization` or `Content-Type`
- Used in session management, personalization, tracking
- consists of *name*, *value*, and (optional) *attributes / flags*
- set with `Set-Cookie` by server, appended with `Cookie` by browser

```
HTTP/1.1 200 OK
Content-type: text/html
Set-Cookie: SESS_ID=9vKnWqiZvuVsIV1zmzJQeYUGINqXYeS; Domain=example.com; Path=/
```


2. Sessions - Cookies: Security

- signed (HMAC) with a secret to mitigate tampering
- *rarely* encrypted (AES) to protected from being read
 - no security concern if read by 3rd party
 - carries no meaningful data (random string)
 - even if encrypted, still a 1-1 match
- encoded (URL) - not for security, but compact (*i.e. compression*)

2. Sessions - Cookies: Attributes

- **Domain** and **Path** (can only be used on a given site & route)
- **Expiration** (can only be used until expiry)
 - when omitted, becomes a *session cookie*
 - gets deleted when browser is closed

2. Sessions - Cookies: Flags

- `HttpOnly` (cannot be read with JS on the client-side)
- `Secure` (can only be sent over encrypted `HTTPS` channel)
- `SameSite` (can only be sent from the same domain, i.e. no CORS sharing)

2. Sessions - Cookies: CSRF

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated.

- mitigated with a CSRF token (e.g. sent in a separate X-CSRF-TOKEN cookie)

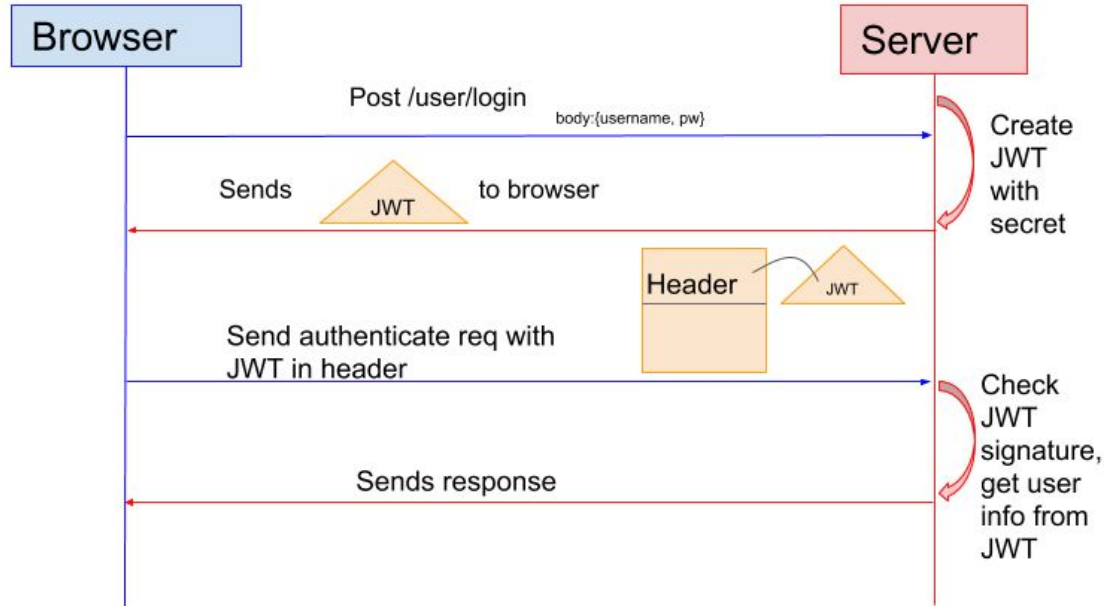
3. Tokens

Flow

- user submits login *credentials*, e.g. email & password
- server verifies the credentials against the DB
- server generates a temporary **token** and embeds user data into it
- server responds back with the token (in body or header)
- user stores the token in client storage
- user sends the token along with each request
- server verifies the token & grants access
- when user logs out, token is cleared from client storage

2. Tokens - Flow

The server creates JWT with a secret and sends the JWT to the client. The client stores the JWT (usually in local storage) and includes JWT in the header with every request. The server would then validate the JWT with every request from the client and sends response.



3. Tokens - Features

- tokens are *not* stored server-side, only on the client (**stateless**)
- *signed* with a secret against tampering
 - verified and can be trusted by the server
- tokens can be *opaque* or *self-contained*
 - carries all required user data in its payload
 - reduces database lookups, but exposes data to XSS (Cross-Site Scripts)
- typically sent in **Authorization** header
- when a token is about to expire, it can be *refreshed*
 - client is issued both access & refresh tokens
- used in SPA web apps, web APIs, mobile apps

3. Tokens - JWT (JSON Web Tokens)

- open standard for authorization & info exchange
- *compact, self-contained, URL-safe* tokens
- signed with *symmetric* (secret) or *asymmetric* (public/private) key

```
HTTP/1.1 200 OK
Content-type: application/json
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1YmQ2MWFhMWJiNDNmNzI0M2EyOTMxNmQiLCJ
uYW1lIjoiSm9obiBTbWl0aCI6Im1hdCI6MTU0MTI3NjA2MH0.WDKey8WGO6LENkHWJRy8S0QOCbdGwFFoH5X
CAR49g4k
```

- contains **header** (meta), **payload** (claims), and **signature** delimited by .

3. Tokens - JWT (JSON Web Tokens)

Header (meta):

```
atob('eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9')  
// {"alg":"HS256","typ":"JWT"}"  
//   ↑ algorithm   ↑ type
```

Payload (claims):

```
atob('eyJzdWIiOiI1YmQ2MWFhMWJlNDNmNzI0M2EyOTMxNmQiLCJuYW1lIjoiSm9obiBTbWl0aCIsIm1hdCI6MTU0MTI3NjA2MH0')  
// {"sub":"5bd61aa1bb43f7243a29316d","name":"John Smith","iat":1541276060}"  
//   ↑ subject (e.g. user ID)           ↑ claims(s)           ↑ issued at (in seconds)
```

Signature: *hashed value*

3. Tokens - JWT: Security

- signed (HMAC) with a secret
 - guarantees that token was not tampered
 - any manipulation (e.g. expiration time) invalidates token
- rarely encrypted (JWE)
 - (web) clients need to read token payload
 - can't store the secret in client storage securely
- encoded (Base64Url) - not for security, but transport
 - payload can be decoded and read
 - no sensitive/private info should be stored
 - access tokens should be short-lived

3. Tokens - JWT: XSS (Cross-Site Scripting)

- client-side script injections
- malicious code can access client storage to:
 - steal user data from the token
 - initiate AJAX (Asynchronous JavaScript) requests on behalf of user
- mitigated by sanitizing & escaping user input

3. Tokens - JWT: Client Storage

- JWT can be stored in client storage, `localStorage` or `sessionStorage`
 - `localStorage` has no expiration time
 - `sessionStorage` gets cleared when page is closed

3. Tokens - JWT: Client Storage

localStorage

Browser key-value store with a simple JS API

Pros

- domain-specific, each site has its own, other sites can't read/write
- max size higher than cookie (5 MB / domain vs. 4 KB / cookie)

Cons

- plaintext, hence not secure by design
- limited to string data, hence need to serialize
- can't be used by web workers
- stored permanently, unless removed explicitly
- accessible to any JS code running on the page (including XSS)
 - scripts can steal tokens or impersonate users

3. Tokens - JWT: Client Storage

Best for

- public, non-sensitive, string data

Worst for

- private sensitive data
- non-string data
- offline capabilities

4. Sessions vs. JWT

Sessions + Cookies

Pros

- session IDs are opaque and carry no meaningful data
- cookies can be secured with flags (same origin, HTTP-only, HTTPS, etc.)
- HTTP-only cookies can't be compromised with XSS exploits
- battle-tested 20+ years in many langs & frameworks

Cons

- server must store each user session in memory
- session auth must be secured against CSRF
- horizontal scaling is more challenging
 - risk of single point of failure
 - need sticky sessions with load balancing

4. Sessions vs. JWT

JWT Auth

Pros

- server does not need to keep track of user sessions
- horizontal scaling is easier (any server can verify the token)
- CORS is not an issue if `Authorization` header is used instead of `Cookie`
- Frontend and Backend architecture is decoupled, can be used with mobile apps
- operational even if cookies are disabled

Cons

- server still has to maintain a blacklist of revoked tokens
 - defeats the purpose of stateless tokens
 - a whitelist of active user sessions is more secure
- when scaling, the secret must be shared between servers
- data stored in token is "cached" and can go *stale* (out of sync)
- tokens stored in client storage are vulnerable to XSS
 - if JWT token is compromised, attacker can
 - steal user info, permissions, metadata, etc.
 - access website resources on user's behalf
- requires JavaScript to be enabled

5. Options for Authorization in SPAs / APIs

1. Sessions
2. Stateless JWT
3. Stateful JWT

5. Options for Authorization in SPAs / APIs

Sessions

- sessions are persisted server-side and linked by session ID
- session ID is signed and stored in a cookie
 - sent via `Set-Cookie` header
 - `HttpOnly`, `Secure`, & `SameSite` flags
 - scoped to the origin with `Domain` & `Path` attributes
- another cookie can hold CSRF token

5. Options for Authorization in SPAs / APIs

Stateless JWT

- user payload embedded in the token
- token is signed & `base64url` encoded
 - sent via `Authorization` header
 - stored in `localStorage` / `sessionStorage` (in plaintext)
- server retrieves user info from the token
- no user sessions are stored server side
- only revoked tokens are persisted
- refresh token sent to renew the access token

5. Options for Authorization in SPAs / APIs

Stateful JWT

- only user ref (e.g. ID) embedded in the token
- token is signed & `base64url` encoded
 - sent as an HTTP-only cookie (`Set-Cookie` header)
 - sent along with non-HTTP `X-CSRF-TOKEN` cookie
- server uses ref. (ID) in the token to retrieve user from the DB
- no user sessions stored on the server either
- revoked tokens still have to be persisted

5. Options for Authorization in SPAs / APIs

Verdict

- Sessions are probably better suited for web apps & websites

Why not JWT?

- server state needs to be maintained either way
- sessions are easily extended or invalidated
- data is secured server side & doesn't leak through XSS
- CSRF is easier to mitigate than XSS (still a concern)
- data never goes stale (always in sync with DB)
- sessions are generally easier to set up & manage
- most apps/sites don't require enterprise scaling

5. Options for Authorization in SPAs / APIs

Auxiliary measures

- IP verification
- user agent verification
- two-factor auth
- API throttling

6. Node modules & Sessions - Features

Features

- login/logout/register + session expiry
- email verification ("Confirm your email")
- password reset ("Forgot password")
- password confirmation ("Re-enter your password")
- persistent login ("Remember me")
- account lockout ("Too many failed login attempts")
- rate limiting ("Too many requests")

6. Node modules & Sessions - Theory

Authentication

- verifying user identity
- public (username/email) + private (pwd, token, 2FA) info

Session Management

- HTTP is a stateless protocol, each request is self-contained
- sessions used to retain user state between requests
- session cookie ties a request to the user's session

Session Timeout

- idle/inactivity: sliding expiration, resets on each request
- absolute: fixed expiration, max duration of lifetime
- renewal: interval until session ID is regenerated

6. Node modules & Sessions - Express

Session Modules

- **express-session**

Session middleware for express. Apply middleware before route handlers. Note Session data is not saved in the cookie itself, just the session ID. Session data is stored server-side.

A Session object is placed as a property into the Request object which is then passed into all other middleware functions and request handler functions.

6. Node modules & Sessions - Session Object

Session object (constructed from parameters via an Options object):

- **cookie object:**
 - domain → default: current domain *(cookie for domain)*
 - path → default: / *(cookie for url path)*
 - expires → default: no expiration *(cookies expiration time)*
 - maxAge → default: none *(cookie's max age in milliseconds)*
 - httpOnly → default: true *(cookie cannot be accessed through client script)*
 - sameSite → default: none *(cookie mitigate against CSRF)*
 - secure → default: false *(cookie requires https)*
- **name property** → default: none **(Session name)**
- **resave property** → default: true **(Force save into session store)**
- **secret property** → required **(Secret used to sign Session ID cookie)**
- **store property** → default: MemoryStore **(Storage for Session data)**

6. Node modules & Sessions - Stores

Session Store Modules

- **connect-redis**, *also requires: ioredis*
- **connect-mongo**, *also requires: mongo*

All Sessions are stored in memory. By default, they are stored as objects in the server's memory. However, there are implementations to store the session data in redis, firebase, mongodb, postgres. If you want to persist them to an actual storage, you can connect them to a redis store for example.

There are many modules used to connect express session to a store:

<https://www.npmjs.com/package/express-session>

Warning: The default server-side session storage, MemoryStore, is purposely not designed for a production environment. It will leak memory under most conditions, does not scale past a single process, and is meant for debugging and developing.

6. Node modules & Sessions - Secured Routes

Local Middleware function

Check if session in request object contains required credentials, if so continue to request handler. Otherwise redirect to another route.

Example:

```
const redirectLogin = (request, response, next) => {  
  if (!request.session.userID){  
    response.redirect('/login');  
  }  
  else{  
    next();  
  }  
}
```

Goal:

Prevent unauthorized requests from accessing "protected" routes that require authentication to view

6. Node modules & Sessions - Hashing Passwords

BCrypt

Middleware for hashing passwords

6. Node modules & Sessions - Input Validation

express-validator

Middleware functions used to validating & sanitizing user inputs. Examples were given in the express lecture.

7. Node modules & JWT

jsonwebtoken library can be used to create the JWT token on the server. Once the user is logged in, the client passes the JWT token back on the header.authorization.bearer attribute.

```
{
  method: "GET",
  headers:{
    "Authorization": "Bearer ${JWT_TOKEN}"
  }
}
```

Middleware, `express-jwt`, can be used to validate the JWT token by comparing the secret.

8. Node Modules & Authentication

- **Auth0**

- Client Side JavaScript toolkit for Auth0 API

- **Passport.js**

- Passport is authentication middleware for Node.js. Extremely flexible and modular, Passport can be unobtrusively dropped in to any Express-based web application. A comprehensive set of strategies support authentication using a username and password, Facebook, Twitter, and more.

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

User Authentication Lab

Homework

Web App with User Logins & Database